

Công cụ hiệu chuẩn, servoing và đo đạc (One-Time Tools)

PHỤ THUỘC B/12-calibration-cho-docking (ba phép hiệu chuẩn và thứ tự bắt buộc của chúng — chương này nói dùng công cụ nào cho từng bước); B/09-multi-marker-constellation mục 5 (đo constellation bằng BA); B/13-docking-control (servoing); C/16-danh-gia-va-nghiem-thu (đánh giá).

TÓM TẮT

Chương này là tăng công cụ cho những gì chạy *một lần* rồi cho ra một tệp tham số hoặc một con số nghiệm thu — khác với D/17 vốn nói về những gì chạy mỗi khung hình. Ranh giới ấy gần trùng với một ranh giới khác đáng chú ý: *thứ bạn nên dùng đồ có sẵn* so với *thứ bạn phải tự cài*. Bốn nhóm công cụ: hiệu chuẩn intrinsic, đo constellation bằng photogrammetry, hand-eye, và đánh giá. Với ba nhóm đầu, lời khuyên đơn giản là dùng công cụ trưởng thành và đừng tự viết. Với nhóm thứ tư thì ngược lại, và lý do đáng nói: các công cụ đánh giá có sẵn được thiết kế cho bài toán quỹ đạo SLAM, còn bài toán docking cần đo *độ lệch tại một điểm* — một đại lượng khác, và tự viết vài chục dòng thì đúng hơn là ép một công cụ không hợp. Nếu chỉ nhớ một điều: *ba bước hiệu chuẩn phải chạy đúng thứ tự, và mỗi bước phải kiểm hiệp phương sai của kết quả trước khi dùng nó cho bước sau*.

KIẾN THỨC CẦN CÓ TRƯỚC

- **photogrammetry** — đo hình học ba chiều từ nhiều ảnh. Với cây này, nó là công cụ thực hiện B/09 mục 5: đo lại vị trí thật của các tag thay vì tin bản vẽ.
- **structure from motion (SfM)** — khôi phục đồng thời cấu trúc 3D và tư thế camera từ một tập ảnh không có thông tin tư thế. **COLMAP** là cài đặt phổ biến nhất [1].
- **gauge fixing** — việc cố định các bậc tự do mà dữ liệu không xác định được — với BA từ ảnh đơn thuần thì đó là vị trí, hướng, và *tỉ lệ* của toàn bộ cấu trúc.
- **ATE / RPE** — *absolute trajectory error* và *relative pose error*: hai chỉ số chuẩn cho đánh giá quỹ đạo SLAM. Chúng *không* phải chỉ số đúng cho docking — mục 5 giải thích.

CÂU HỎI MỞ ĐẦU

- Q1. Ba bước hiệu chuẩn của B/12 cần ba công cụ khác nhau. Nêu công cụ cho từng bước, và nói bước nào bạn có thể phải tự viết.
- Q2. Bạn dùng **COLMAP** để đo constellation. Nó trả về một cấu trúc 3D đẹp. Vì sao bạn *vẫn chưa* có thứ cần dùng, và còn thiếu một thông tin gì?
- Q3. Các công cụ đánh giá quỹ đạo như **evo** rất trưởng thành. Vì sao chúng lại *không* phải công cụ đúng cho nghiệm thu docking?
- Q4. Sau khi chạy BA đo constellation, có một đại lượng bạn phải nhìn trước khi dùng kết quả — và bỏ qua nó là tự lừa mình. Đại lượng nào, và tự lừa mình thế nào?
- Q5. Trong bốn nhóm công cụ của chương, ba nhóm nên dùng đồ có sẵn và một nhóm nên tự viết. Nhóm nào, và nguyên tắc chung rút ra là gì?

1.1 Đặt vấn đề

D/17 nói về những gì chạy mỗi khung hình. Chương này nói về những gì chạy một lần: hiệu chuẩn, đo đạc, và đánh giá.

Ranh giới ấy quan trọng hơn vẻ ngoài, vì nó gần trùng với một ranh giới khác về mặt kỹ thuật phần mềm. Thứ chạy mỗi khung phải nhanh, phải chạy trên máy của robot, và phải tích hợp chặt với pipeline — nên bạn thường phải tự cài hoặc ít nhất tự sửa. Thứ chạy một lần thì chạy trên máy tính bàn, không có ràng buộc thời gian thực, và nó cho ra một tệp — nên dùng công cụ có sẵn gần như luôn đúng.

“Gần như” vì có một ngoại lệ, và nó là nội dung đáng nói nhất của chương. Công cụ đánh giá có sẵn được thiết kế cho bài toán *quỹ đạo SLAM*: chúng so hai quỹ đạo dài, có các phép căn chỉnh tinh vi, và chúng báo cáo ATE và RPE. Bài toán docking không phải bài toán ấy — nó cần đo *độ lặp lại của một điểm cuối* qua nhiều lần thử. Ép một công cụ quỹ đạo làm việc ấy thì được, nhưng nó phức tạp hơn và dễ sai hơn so với vài chục dòng tự viết. Mục 5 bàn kỹ.

Có một lý do thứ hai để chương này tồn tại, và nó thuộc về quy trình chứ không về công cụ: *thứ tự*. B/12 mục 2 chỉ ra rằng ba phép hiệu chuẩn phải chạy theo đúng thứ tự vì mỗi bước dùng kết quả bước trước làm hằng số. Chương này bổ sung phần thực hành: mỗi bước dùng công cụ nào, và — quan trọng hơn — mỗi bước phải kiểm gì trước khi chuyển sang bước sau. Bỏ qua bước kiểm ấy là cách phổ biến nhất để có một chuỗi hiệu chuẩn trông hoàn chỉnh mà sai.

1.2 Hiệu chuẩn intrinsic

Bước một của B/12 mục 2.

`OpenCV` `calibrateCamera` — đủ dùng cho phần lớn trường hợp, có sẵn, và nó cài phương pháp Zhang [2]. Nhược điểm: nó không cho bạn bản đồ residual theo vị trí ảnh, mà đó là phép kiểm quan trọng nhất (A/01 mục 6). Bạn phải tự vẽ — vài chục dòng, và nó đáng.

`Kalibr` [3] — mạnh hơn, hỗ trợ nhiều mô hình méo, và nó có báo cáo residual theo vị trí. Nó cũng xử lý được nhiều camera và camera-IMU. Giá phải trả: nặng hơn để cài đặt, và nó được thiết kế quanh quy trình ROS bag.

`deltille` [4] — không phải công cụ hiệu chuẩn hoàn chỉnh mà là một detector target chất lượng cao. Dùng nó khi cần intrinsic chính xác nhất và khi target lưới tam giác chấp nhận được.

Về target: B/12 mục 3C khuyến nghị ChArUco hoặc `deltille`, và nhấn mạnh rằng target phải *phẳng thật* — in trên giấy dán bìa cứng không đủ. Với hiệu chuẩn chất lượng cao, target in trên tấm nhôm hoặc kính là đầu tư đáng.

Phải kiểm trước khi chuyển sang bước hai: bản đồ residual không có cấu trúc (A/01 mục 6), và c_x, c_y ổn định qua các tập con ảnh (B/12 mục 3B). Nếu điểm chính nhảy hàng chục pixel giữa các tập con thì bạn chưa có intrinsic dùng được, và chạy bước hai với nó là lãng phí.

1.3 Đo constellation bằng photogrammetry

Bước hai, và mục này trả lời Q2 và Q4.

A. Ba lựa chọn

Tự viết BA bằng [Ceres](#) [5] hoặc [GTSAM](#) [6]. Đây là cách cho kết quả tốt nhất, vì bạn tận dụng được cấu trúc mà một công cụ tổng quát không biết: bốn góc của một tag *cứng với nhau* theo một hình vuông đã biết cạnh, nên mỗi tag chỉ có sáu bậc tự do chứ không phải mười hai. Ràng buộc ấy giảm mạnh số ẩn và cải thiện điều kiện số. Chi phí: vài trăm dòng.

[COLMAP](#) [1]. Đường tắt thực dụng: nó là một hệ SfM hoàn chỉnh, nó xử lý được ảnh không có thứ tự, và nó trưởng thành. Nhược điểm là nó được thiết kế cho đặc trưng tự nhiên chứ không cho marker, nên bạn phải đưa các góc tag vào như các điểm đã khớp sẵn — làm được nhưng không phải luồng chính của nó.

[tagslam](#) [7]. Được thiết kế đúng cho bài toán này: nó giải factor graph trên nhiều tag và ước lượng cả vị trí tag. Nếu nó chạy được với thiết lập của bạn thì nó là lựa chọn hợp lý nhất. Mức tin C — tiền ẩn — nhưng có mã nguồn và được dùng rộng rãi.

B. Điều còn thiếu sau khi chạy

Đây là câu trả lời cho Q2, và nó là chỗ dễ bỏ sót nhất trong cả chuỗi.

Kết quả BA là tọa độ các góc tag trong một hệ tọa độ *tùy ý* do thuật toán chọn, và nó xác định sai khác một hệ số *tỉ lệ*. Hai thông tin còn thiếu.

Một, tỉ lệ. Ảnh đơn thuần không xác định được kích thước tuyệt đối. Phải neo bằng một khoảng cách đo được — và B/09 mục 5B khuyến nghị đo cạnh của tag *lớn nhất*, vì sai số của phép neo trở thành sai số tỉ lệ của toàn bộ constellation. Đây là chỗ duy nhất trong cả quy trình mà thước cặp còn cần thiết.

Hai, và nó hay bị quên hơn: quan hệ với điểm cập chuẩn. B/12 mục 4 gọi nó là khâu $T_{\text{dock}}^{\text{tag}}$. BA cho bạn các tag ở đâu so với nhau; nó không cho bạn cụm tag ấy ở đâu so với cái lỗ cắm sạc. Cách đo đúng và rẻ nhất, như B/12 mục 4 nêu: đưa robot vào vị trí cập chuẩn rồi đo pose constellation từ đó — pose ấy *định nghĩa* điểm cập chuẩn, không cần thước.

C. Đại lượng phải nhìn trước khi dùng kết quả

Đây là câu trả lời cho Q4: *hiệp phương sai của tọa độ các góc*, mà BA trả về như một sản phẩm phụ.

Cách tự lừa mình nếu bỏ qua nó: bạn thay một con số không biết sai bao nhiêu (từ CAD) bằng một con số khác cũng không biết sai bao nhiêu (từ BA), rồi tin con số thứ hai hơn vì nó “đã được đo”. Nếu BA trả về bất định 2 mm cho một tag — chuyện hoàn toàn có thể khi ảnh ít hoặc góc nhìn nghèo — thì bạn chưa cải thiện gì so với CAD, và bạn đã tiêu một buổi để tin sai.

Phép kiểm cụ thể: so hiệp phương sai BA với dung sai gia công dự kiến. Nếu nó không nhỏ hơn đáng kể thì phải chụp thêm ảnh, đặc biệt là ảnh phủ các tag có bất định lớn từ nhiều góc khác nhau.

Phải kiểm trước khi chuyển sang bước ba: hiệp phương sai đủ nhỏ, và sai số tái chiếu của bài toán hợp nhất không lớn hơn hẳn so với khi giải từng tag riêng (B/09 mục 2).

1.4 Hand-eye

Bước ba.

[OpenCV](#) `calibrateHandEye` — cài nhiều phương pháp chọn được: Tsai–Lenz [8], Park–Martin [9], Daniilidis [10], và vài cái khác. Một mẹo chẩn đoán tốt: *chạy với vài phương pháp khác nhau và so kết quả*. Chúng giải cùng một bài toán bằng các đường khác nhau, nên nếu dữ liệu đủ kích thích

thì chúng phải cho kết quả gần nhau. Nếu chúng khác nhau đáng kể thì dữ liệu của bạn không đủ — và đó là chỉ báo rõ nhất cho điều kiện kích thích ở B/12 mục 5B.

`easy_handeye` — wrapper ROS với quy trình thu thập dữ liệu. Tiện, nhưng nó không giải quyết được chuyện điều kiện kích thích: nếu robot của bạn chỉ xoay quanh một trục thì thành phần độ cao vẫn không quan sát được bất kể công cụ nào.

Với robot vi sai, nhớ ba cách xử lý ở B/12 mục 5B — và nhớ rằng cách thứ ba (nhận ra rằng bạn không cần nó) thường là cách đúng.

Phải kiểm sau bước ba: kết quả ổn định qua các tập con dữ liệu, và các phương pháp khác nhau cho kết quả gần nhau.

1.5 Đánh giá: chỗ nên tự viết

Mục này trả lời Q3 và Q5, và nó là mục có nội dung riêng nhất của chương.

`evo` [11] và các công cụ tương tự là công cụ tốt cho việc chúng được thiết kế: so hai quỹ đạo. Chúng có các phép căn chỉnh chuẩn, chúng tính ATE và RPE, và chúng vẽ đồ thị đẹp. Tài liệu tham chiếu về cách dùng chúng cho đúng là [12].

Vì sao chúng không phải công cụ đúng cho nghiệm thu docking — câu trả lời cho Q3: vì bài toán khác về bản chất.

Bài toán quỹ đạo hỏi: hai đường đi này khác nhau bao nhiêu, sau khi đã căn chỉnh tối ưu? Nó có một chuỗi dài các pose và nó quan tâm tới sai số tích lũy.

Bài toán docking hỏi: qua ba mươi lần cập, điểm cuối tản mát bao nhiêu, và nó có lệch so với vị trí đúng không? Nó chỉ quan tâm tới một pose — pose cuối — nhưng qua nhiều lần thử.

Hai câu hỏi ấy cần hai phép tính khác nhau, và phép tính thứ hai đơn giản hơn nhiều: gom các pose cuối, tính trung bình và độ tản mát, và nếu có chuẩn tham chiếu thì so trung bình với nó. Vài chục dòng, và mọi bước đều minh bạch.

Ép `evo` làm việc ấy thì được — coi mỗi lần cập là một “quỹ đạo” một điểm — nhưng bạn phải hiểu rõ nó đang căn chỉnh cái gì, và các phép căn chỉnh của nó được thiết kế cho tình huống khác. Rủi ro không phải là nó không chạy mà là nó chạy và cho một con số mà bạn diễn giải sai.

Nguyên tắc chung rút ra — câu trả lời cho Q5: dùng công cụ có sẵn khi bài toán của bạn đúng là bài toán mà nó được thiết kế cho; tự viết khi bài toán khác, kể cả khi công cụ ấy “làm được”. Ba nhóm đầu của chương thuộc trường hợp thứ nhất — hiệu chuẩn intrinsic, BA, hand-eye đều là đúng bài toán mà công cụ giải. Nhóm thứ tư thuộc trường hợp thứ hai.

Và một lưu ý về cái gì phải đo, lấy từ C/16 mục 4B: đo độ lặp lại của toàn bộ chu trình — lùi ra hẳn rồi cập lại — chứ không đo độ tản mát của pose khi robot đã đứng yên ở vị trí cuối. Hai con số ấy khác nhau nhiều, và chỉ con số thứ nhất có nghĩa với người dùng.

1.6 Giới hạn

Các khẳng định trong chương này chưa được chạy kiểm. Khác với D/17 mục 3B, nơi tôi chạy thật trên OpenCV và phát hiện một cái bẫy, chương này mô tả khả năng của các công cụ từ hiểu biết chung về chúng. Trước khi dựa vào một khẳng định cụ thể — chẳng hạn rằng `calibrateHandEye` có đúng những phương pháp tôi liệt kê — hãy kiểm với tài liệu của phiên bản bạn dùng.

Không có benchmark. Chương không nói công cụ nào cho kết quả chính xác hơn, vì tôi không có dữ liệu so sánh được.

Chốt tháng 9/2026. Cùng cảnh báo như D/17 mục 1.

tagslam ở mức tin C. Tiềm ẩn, chưa qua phản biện; tôi đưa nó vào vì nó có mã nguồn công khai và giải đúng bài toán, nhưng theo `research-rules.md` mục 4 thì nguồn mức C không được đứng một mình làm chỗ dựa.

1.7 Thực hành

Quy trình ba bước, với phép kiểm sau mỗi bước — và các phép kiểm mới là phần đáng giá.

Bước một, intrinsic. Công cụ: OpenCV hoặc Kalibr. Kiểm: bản đồ residual không có cấu trúc; c_x, c_y ổn định qua các tập con.

Bước hai, constellation. Công cụ: Ceres/GTSAM tự viết, hoặc COLMAP, hoặc tagslam. Kiểm: hiệp phương sai đủ nhỏ so với dung sai gia công; và đừng quên neo tỉ lệ cùng khâu $T_{\text{dock}}^{\text{tag}}$.

Bước ba, hand-eye. Công cụ: OpenCV `calibrateHandEye`. Kiểm: các phương pháp khác nhau cho kết quả gần nhau; kết quả ổn định qua các tập con.

Và cho đánh giá: tự viết, theo C/16 mục 7.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ranh giới của chương này với D/17	D/17: thứ chạy <i>mỗi khung</i> (detector, PnP) — thường phải tự cài hoặc tự sửa. Chương này: thứ chạy <i>một lần</i> (hiệu chuẩn, đo đạc, đánh giá) — thường nên dùng đồ có sẵn.
••• Ba bước và ba công cụ	Intrinsic: OpenCV <code>calibrateCamera</code> hoặc Kalibr . Constellation: Ceres/GTSAM tự viết, hoặc COLMAP , hoặc tagslam . Hand-eye: OpenCV <code>calibrateHandEye</code> . <i>Thứ tự bắt buộc</i> theo B/12 mục 2.
••• Vì sao tự viết BA tốt hơn công cụ tổng quát	Vì bạn tận dụng được ràng buộc mà công cụ tổng quát không biết: bốn góc một tag <i>cứng với nhau</i> theo hình vuông đã biết cạnh, nên mỗi tag có 6 bậc tự do chứ không phải 12. Giảm mạnh số ẩn, cải thiện điều kiện số.
••• Hai thứ còn thiếu sau khi chạy BA	<i>Tỉ lệ</i> — ảnh đơn thuần không xác định kích thước tuyệt đối, phải neo bằng cạnh tag <i>lớn nhất</i> . Và khâu $T_{\text{dock}}^{\text{tag}}$ — BA cho các tag ở đâu so với nhau, không cho cụm ấy ở đâu so với lỗ cấm.
••• Đại lượng phải nhìn sau BA	<i>Hiệp phương sai của tọa độ các góc</i> . Bỏ qua nó là tự lừa mình: thay một con số không biết sai bao nhiêu (CAD) bằng một con số khác cũng không biết sai bao nhiêu (BA), rồi tin cái thứ hai hơn vì nó “đã được đo”.
•• Phép kiểm rẻ nhất cho hand-eye	Chạy với <i>vài phương pháp khác nhau</i> (Tsai–Lenz, Park–Martin, Daniilidis) và so. Chúng giải cùng bài toán bằng các đường khác nhau, nên dữ liệu đủ kích thích thì chúng phải gần nhau. Khác nhau đáng kể $\Rightarrow$ dữ liệu chưa đủ.
••• Vì sao evo không đúng cho docking	Bài toán khác về bản chất. Quỹ đạo: hai đường đi khác nhau bao nhiêu sau khi căn chỉnh tối ưu, quan tâm sai số tích lũy. Docking: qua 30 lần cặp, <i>một</i> pose cuối tằn mết bao nhiêu. Hai câu hỏi, hai phép tính.
••• Nguyên tắc chung về chọn công cụ	Dùng đồ có sẵn khi bài toán của bạn <i>đúng là</i> bài toán nó được thiết kế cho; tự viết khi bài toán khác — <i>kể cả khi công cụ ấy “làm được”</i> . Rủi ro không phải nó không chạy mà là nó chạy và bạn diễn giải sai.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
•• Do cái gì khi nghiệm thu	Độ lặp lại của <i>toàn bộ chu trình</i> — lùi ra hẳn rồi cập lại — chứ không phải độ tần mắt của pose khi robot đã đứng yên ở vị trí cuối. Hai con số khác nhau nhiều (C/16 mục 4B).
•• Phép kiểm sau mỗi bước hiệu chuẩn	Sau intrinsic: bản đồ residual không cấu trúc, c_x, c_y ổn định. Sau constellation: hiệp phương sai đủ nhỏ, residual hợp nhất không lớn hơn hẳn từng tag. Sau hand-eye: các phương pháp cho kết quả gần nhau.
• Chỗ duy nhất thước cặp còn cần	Neo tỉ lệ cho BA — đo cạnh của tag <i>lớn nhất</i> , vì sai số của phép neo thành sai số tỉ lệ toàn cục.
• Mức kiểm chứng của chương này	Các khẳng định <i>chưa được chạy kiểm</i> trong lần soạn này, khác với D/17 mục 3B. Kiểm với tài liệu của phiên bản bạn dùng trước khi dựa vào một khẳng định cụ thể.

••• phải thuộc •• nên nắm • biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
Kalibr [3]	Hiệu chuẩn nhiều mô hình méo, có báo cáo residual theo vị trí ảnh	Công cụ duy nhất mặc định cho bạn phép kiểm quan trọng nhất của A/01 mục 6
Ceres [5] / GTSAM [6]	Tự viết BA với ràng buộc tag cứng — cho kết quả tốt nhất	Vài trăm dòng; đáng nếu constellation là dòng lớn trong ngân sách
COLMAP [1]	SfM hoàn chỉnh, đường tắt cho đo constellation	Thiết kế cho đặc trưng tự nhiên; đưa góc tag vào như điểm đã khớp
OpenCV calibrateHandEye	Nhiều phương pháp chọn được [8]–[10]	Chạy vài phương pháp và so — chỉ báo rõ nhất cho điều kiện kích thích
evo [11]	Đánh giá quỹ đạo với các phép căn chỉnh chuẩn [12]	Tốt cho quỹ đạo, <i>không</i> phải công cụ đúng cho độ lặp lại điểm cuối

MINI PROJECT · Chạy chuỗi ba bước hiệu chuẩn với phép kiểm sau mỗi bước · 3–4 ngày

Mục tiêu	có một chuỗi hiệu chuẩn hoàn chỉnh mà mỗi bước đã được <i>kiểm</i> chứ không chỉ đã <i>chạy</i> — khác biệt quyết định giữa một tệp tham số dùng được và một tệp trông như dùng được.
Dữ liệu	phần cứng thật: camera, target hiệu chuẩn phẳng chất lượng, constellation đã lắp, và robot.
Các bước	bước một, hiệu chuẩn intrinsic với ít nhất 30 ảnh có tư thế nghiêng mạnh và phủ tới bốn góc khung hình; vẽ bản đồ residual và chạy lại với các tập con để kiểm c_x, c_y ; <i>không chuyển bước</i> cho tới khi cả hai phép kiểm đạt. Bước hai, chụp ít nhất 30 ảnh constellation từ nhiều góc, chạy BA, neo tỉ lệ bằng cạnh tag lớn nhất, và lấy hiệp phương sai của kết quả; so nó với dung sai gia công; rồi đo khâu $T_{\text{dock}}^{\text{tag}}$ bằng cách đưa robot vào vị trí cập chuẩn. Bước ba, thu thập cặp pose cho hand-eye, chạy với ba phương pháp khác nhau, so kết quả.
Xong khi	bạn có một tệp cấu hình với ba nhóm tham số, và <i>cho mỗi nhóm</i> một con số bất định kèm theo. Kiểm tra chéo bắt buộc: chạy lại toàn bộ chuỗi từ đầu với một bộ ảnh mới và so hai kết quả — nếu chúng khác nhau nhiều hơn bất định đã ước lượng, thì bất định của bạn đang bị đánh giá thấp, và mọi dòng hiệu chuẩn trong bảng ngân sách của C/15 cũng đang bị đánh giá thấp theo.

Nối với

C/15 nhận ba con số bất định làm ba dòng của bảng; B/12 mini project là phiên bản sâu hơn của phép kiểm ổn định.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Intrinsic: [OpenCV](#) `calibrateCamera` hoặc [Kalibr](#) — cái sau đáng hơn vì nó báo cáo residual theo vị trí ảnh, phép kiểm quan trọng nhất. Constellation: [Ceres](#) hoặc [GTSAM](#) tự viết, hoặc [COLMAP](#), hoặc [tag slam](#). Hand-eye: [OpenCV](#) `calibrateHandEye`. Bước có thể phải tự viết là bước hai, và lý do đáng nói: một công cụ SfM tổng quát không biết rằng bốn góc của một tag *cứng với nhau* theo một hình vuông đã biết cạnh, nên nó xử lý chúng như bốn điểm độc lập — mười hai bậc tự do thay vì sáu. Tự viết cho phép áp ràng buộc ấy, giảm mạnh số ẩn và cải thiện điều kiện số. Với constellation nhỏ thì chênh lệch có thể không đáng, nhưng nếu constellation là dòng lớn trong ngân sách của bạn thì vài trăm dòng code là đầu tư đúng.
- A2. Vì kết quả BA là toạ độ trong một hệ *tuyệt đối* và xác định sai khác một hệ *số tỉ lệ* — ảnh đơn thuần không bao giờ xác định được kích thước tuyệt đối. Thông tin còn thiếu thứ nhất là phép neo tỉ lệ: một khoảng cách đo được bằng thước, và nên đo cạnh của tag *lớn nhất* vì sai số của phép neo trở thành sai số tỉ lệ của toàn bộ constellation. Thông tin còn thiếu thứ hai, và nó hay bị quên hơn: quan hệ giữa hệ constellation và *điểm cập chuẩn thật* — khâu T_{dock}^{tag} của B/12 mục 4. BA cho bạn biết các tag ở đâu so với nhau; nó hoàn toàn không biết cái lỗ cắm sặc ở đâu. Cách đo khâu ấy rẻ và đẹp: đưa robot vào vị trí cập đúng rồi đo pose constellation từ đó — pose ấy *định nghĩa* điểm cập chuẩn, và không cần thước nào cả.
- A3. Vì bài toán khác về bản chất chứ không chỉ về quy mô. [evo](#) được thiết kế cho bài toán *quỹ đạo*: so hai đường đi dài sau khi căn chỉnh tối ưu, với các chỉ số ATE và RPE quan tâm tới sai số tích lũy. Bài toán docking hỏi một câu khác hẳn: qua ba mươi lần cập độc lập, *một* pose — pose cuối — tằn mát bao nhiêu, và nó có lệch so với vị trí đúng không? Đó là một bài toán thống kê trên nhiều lần thử ở một điểm, không phải một bài toán so sánh hai chuỗi. Phép tính đúng đơn giản hơn nhiều: gom các pose cuối, tính trung bình và độ tằn mát, so trung bình với chuẩn tham chiếu nếu có. Ép [evo](#) làm việc ấy thì chạy được, nhưng rủi ro không phải là nó không chạy — rủi ro là nó chạy và cho một con số mà bạn diễn giải sai, vì bạn không biết chính xác nó đã căn chỉnh cái gì.
- A4. Hiệp phương sai của toạ độ các góc, mà BA trả về như một sản phẩm phụ. Cách tự lừa mình nếu bỏ qua nó: bạn bắt đầu với một con số từ CAD mà bạn không biết sai bao nhiêu, bạn chạy BA và được một con số khác mà bạn *cũng* không biết sai bao nhiêu, rồi bạn tin con số thứ hai hơn vì nó “đã được đo”. Nhưng nếu BA trả về bất định 2 mm cho một tag — hoàn toàn có thể khi số ảnh ít, góc nhìn nghèo, hoặc intrinsic chưa tốt — thì bạn chưa cải thiện gì so với CAD, và bạn vừa tiêu một buổi để có được sự tự tin sai chỗ. Phép kiểm cụ thể: so hiệp phương sai BA với dung sai gia công dự kiến; nếu nó không nhỏ hơn đáng kể thì phải chụp thêm ảnh, đặc biệt là ảnh phủ các tag có bất định lớn từ nhiều góc khác nhau.
- A5. Nhóm *đánh giá* nên tự viết; ba nhóm còn lại — hiệu chuẩn intrinsic, đo constellation, hand-eye — nên dùng đồ có sẵn. Nguyên tắc chung: *dùng công cụ có sẵn khi bài toán của bạn đúng là bài toán mà nó được thiết kế cho, và tự viết khi bài toán khác — kể cả khi công cụ ấy “làm được”*. Ba nhóm đầu thuộc trường hợp thứ nhất: hiệu chuẩn camera là đúng bài toán mà `calibrateCamera` giải, BA là đúng bài toán mà `Ceres` giải. Nhóm thứ tư thuộc trường hợp thứ hai: [evo](#) giải bài toán quỹ đạo, còn bạn có bài toán độ lặp lại một điểm. Điều làm nguyên tắc này đáng nhớ là chỗ “*kể cả khi làm được*”: một công cụ ép vào bài toán sai vẫn cho ra số, và số ấy trông hợp lệ — đó mới là rủi ro, chứ không phải chuyện nó báo lỗi.

NÓI LẠI THẬT ĐƠN GIẢN

Chương trước nói về mấy thứ chạy liên tục khi robot đang làm việc. Chương này nói về mấy thứ chạy đúng một lần rồi cho ra một tệp con số.

Có ba việc như vậy, và chúng phải làm đúng thứ tự: đo cái camera trước, rồi đo xem mấy tấm tag thật sự nằm đâu, rồi cuối cùng đo xem cái camera được bắt ở chỗ nào trên robot. Làm ngược thì sai số của bước trước chui vào bước sau và không còn tách ra được.

Điều quan trọng nhất của chương không phải tên công cụ mà là: *sau mỗi bước phải kiểm, chứ không phải chạy xong là tin*. Và phép kiểm thường rất đơn giản — chạy lại với một nửa dữ liệu rồi xem kết quả có đổi không. Con số nào giữ nguyên thì tin được; con số nào nhảy thì nó chưa được xác định, dù phần mềm có in nó ra với năm chữ số thập phân.

Riêng bước đo mấy tấm tag có một cái bẫy đáng nhớ. Bạn chụp mấy chục tấm ảnh, chạy chương trình, và nó cho ra một mô hình ba chiều rất đẹp. Nhưng mô hình ấy chưa dùng được, vì nó thiếu hai thứ. Thứ nhất: ảnh không biết kích thước thật — một cái dock nhỏ chụp gần và một cái dock lớn chụp xa cho ảnh giống hệt nhau. Phải đo một cái cạnh bằng thước để neo lại, và đây là chỗ duy nhất trong cả quy trình mà cây thước còn cần thiết. Thứ hai, và hay quên hơn: chương trình cho biết mấy tấm tag nằm đâu *so với nhau*, nhưng nó hoàn toàn không biết cái lỗ cắm sạc nằm đâu. Cách đo cái đó thì đẹp: cứ đẩy robot vào cắm cho đúng, rồi chụp một tấm — thế là xong, không cần đo gì cả.

Cuối cùng là chuyện chọn công cụ, và có một nguyên tắc đáng nhớ. Dùng đồ có sẵn khi việc của bạn đúng là việc mà người ta làm ra nó để giải quyết. Tự viết khi việc của bạn khác — kể cả khi cái công cụ kia “cũng làm được”. Chỗ nguy hiểm không phải là công cụ báo lỗi; chỗ nguy hiểm là nó chạy trơn tru và cho ra một con số mà bạn hiểu nhầm.

Ví dụ cụ thể: có những công cụ rất tốt để so hai đường đi của robot. Việc của bạn thì khác — bạn cần biết qua ba mươi lần cập dock, cái điểm cuối tảo mất bao nhiêu. Ép cái công cụ kia làm việc này thì được, nhưng viết ba chục dòng tự tính thì đúng hơn và bạn hiểu rõ mình đang tính gì.

Chỗ không rút gọn được: không có cách nào biết trước một công cụ có phù hợp hay không ngoài việc hiểu rõ nó được thiết kế để giải bài toán nào. Tài liệu thường nói nó *làm được gì*, ít khi nói nó *được thiết kế cho gì* — và hai điều đó không giống nhau.

Một câu: *Chạy xong chưa phải xong — chạy lại với nửa dữ liệu rồi xem con số nào nhảy, đó mới là kiểm.*

1.8 Điều gì chứng minh cách hiểu này sai

Chương này có mức kiểm chứng *thấp hơn D/17*, và mục 6 đã ghi rõ lý do: tôi không chạy các công cụ trong lần soạn này. Vì vậy mục này liệt kê những khẳng định đáng nghi nhất.

Khẳng định có hậu quả lớn nhất — *tự viết BA với ràng buộc tag cứng cho kết quả tốt hơn dùng một công cụ SfM tổng quát* — là suy luận của tôi từ việc đếm bậc tự do (sáu thay vì mười hai cho mỗi tag). Nó bị bác bỏ nếu một so sánh thật cho thấy COLMAP với các góc tag đưa vào như điểm đã khớp đạt cùng độ chính xác. Điều đó hoàn toàn có thể khi số ảnh lớn, vì khi ấy thông tin dư thừa bù được cho ràng buộc bị bỏ qua — và nếu vậy thì lời khuyên “tự viết” là lời khuyên tốn công vô ích. Đây là mệnh đề đáng kiểm tra trước khi bỏ ra vài trăm dòng.

Khẳng định thứ hai — *chạy hand-eye với nhiều phương pháp rồi so là chỉ báo cho điều kiện kích thích* — dựa trên lập luận rằng các phương pháp giải cùng bài toán bằng các đường khác nhau nên phải hội tụ khi dữ liệu đủ. Nó bị bác bỏ nếu hai phương pháp cho kết quả khác nhau *ngay cả khi* dữ liệu đủ kích thích, chẳng hạn vì chúng xử lý nhiễu theo những cách khác nhau. Trong trường hợp ấy phép thử cho báo động giả, và phép thử đúng phải là chạy lại với các tập con dữ liệu.

Khẳng định thứ ba — *công cụ đánh giá quỹ đạo không phù hợp cho nghiệm thu docking* — là một mệnh đề về sự phù hợp của công cụ với bài toán, không phải một mệnh đề thực nghiệm. Nó “bị bác bỏ” nếu ai đó chỉ ra một cách dùng *evo* cho bài toán độ lặp lại điểm cuối mà vừa đúng vừa đơn giản hơn vài chục dòng tự viết. Tôi không biết cách nào, nhưng tôi cũng không khẳng định là không có.

1.9 Kết nối

Nối lên trên. B/12-calibration-cho-docking là chương lý thuyết tương ứng: nó nói vì sao ba bước phải theo thứ tự ấy và phải kiểm gì; chương này nói dùng công cụ nào. B/09-multi-marker-constellation mục 5 là nguồn của mục 3.

Nối ngang. D/17-thu-vien-va-detector là chương anh em, chia theo ranh giới mỗi-khung so với một-lần.

Nối xuống. C/15-ngan-sach-sai-so nhận ba con số bất định từ mini project của chương này làm ba dòng của bảng. C/16-danh-gia-va-nghiem-thu mục 7 là nơi mục 5 dẫn tới: quy trình nghiệm thu tự viết.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Nêu ba bước hiệu chuẩn, công cụ cho từng bước, và phép kiểm bắt buộc sau mỗi bước.
2. Vì sao tự viết BA có thể tốt hơn dùng một công cụ SfM tổng quát? Nêu ràng buộc mà công cụ tổng quát không biết.
3. Sau khi BA chạy xong, bạn còn thiếu hai thông tin. Nêu cả hai, và nêu cách đo thông tin thứ hai mà không cần thước.
4. Vì sao bỏ qua hiệp phương sai của kết quả BA là tự lừa mình? Mô tả cơ chế.
5. Nêu phép kiểm rẻ nhất cho điều kiện kích thích của hand-eye.
6. Vì sao công cụ đánh giá quỹ đạo không phải công cụ đúng cho nghiệm thu docking, và rủi ro cụ thể của việc ép nó là gì?
7. Phát biểu nguyên tắc chung về khi nào dùng đồ có sẵn và khi nào tự viết.

Nguồn

- [1] J. L. Schönberger and J.-M. Frahm, “Structure-from-Motion Revisited,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; COLMAP, 2016.
- [2] Z. Zhang, “A Flexible New Technique for Camera Calibration,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334, 2000, mức tin A; phương pháp hiệu chuẩn từ target phẳng mà mọi hộp công cụ đang dùng.
- [3] P. Furgale, J. Rehder, and R. Siegwart, “Unified Temporal and Spatial Calibration for Multi-Sensor Systems,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; nền của Kalibr, 2013.
- [4] H. Ha, M. Perdoch, H. Alismail, et al., “Deltile Grids for Geometric Camera Calibration,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A; lưới tam giác và tinh chỉnh saddle point bậc hai, chính xác hơn checkerboard, 2017.
- [5] S. Agarwal, K. Mierle, et al., “Ceres Solver,” <http://ceres-solver.org>, 2012, mức tin A; phần mềm, không phải bài báo.
- [6] F. Dellaert and M. Kaess, “Factor Graphs for Robot Perception,” *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017, mức tin A.
- [7] B. Pfrommer and K. Daniilidis, “TagSLAM: Robust SLAM with Fiducial Markers,” in *arXiv preprint arXiv:1910.00679*, mức tin C; tiền ấn nhưng có mã nguồn công khai và được dùng rộng rãi — dùng làm gợi ý về kiến trúc, không làm bằng chứng số, 2019.
- [8] R. Y. Tsai and R. K. Lenz, “A New Technique for Fully Autonomous and Efficient 3D Robotics Hand/Eye Calibration,” *IEEE Transactions on Robotics and Automation*, vol. 5, no. 3, pp. 345–358, 1989, mức tin A; lời giải kinh điển cho $AX=XB$.
- [9] F. C. Park and B. J. Martin, “Robot Sensor Calibration: Solving $AX=XB$ on the Euclidean Group,” *IEEE Transactions on Robotics and Automation*, vol. 10, no. 5, pp. 717–721, 1994, mức tin A.
- [10] K. Daniilidis, “Hand-Eye Calibration Using Dual Quaternions,” *The International Journal of Robotics Research*, vol. 18, no. 3, pp. 286–298, 1999, mức tin A; giải đồng thời xoay và tịnh tiến.
- [11] M. Grupp, “evo: Python Package for the Evaluation of Odometry and SLAM,” <https://github.com/MichaelGrupp/evo>, 2017, mức tin B; phần mềm.
- [12] Z. Zhang and D. Scaramuzza, “A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; căn chỉnh quỹ đạo trước khi tính sai số, và vì sao chọn sai phép căn chỉnh làm con số vô nghĩa, 2018.